Project 2 Manual

CPU Name: Mini CPU
Names: Tracy Jiang, Jamie Lu

Job Description:

- Tracy:
    - Built the cpu in Logisim
    - Debugging the cpu
- Jamie:
    - Wrote the assembly test file and python compiler program
    - Wrote this manual
    - Debugging the cpu

How to Use Assembler Program:

1. Go to a terminal
2. Find the location (directory) of the assembler program (named "translateAssembly.py")
3. Once you're in the correct directory, run the program:
   python translateAssembly.py
4. Afterwards it will produce the following two image .txt files in the same directory:
    - ❖ instructionMemory.txt
    - ❖ dataMemory.txt
5. To import it, you would have to right click each ram in the cpu and load the respective image files. Now you should have both the instruction and data memory loaded starting at address 00.

Architecture Description:

- We have 4 general registers in our Register Files (X0, X1, X2, X3)
- 2 rams (1 for Instruction Memory and 1 for Data Memory)
- An ALU for arithmetic instructions
- PC counter
- Instructions:
    - ADD
    - SUB
    - LDR

    - STR
- Each instructions use 8 bits

Instructions:

ADD Rd, Rn, Rm

Adds 2 registers (Rn & Rm) together and the sum gets stored in the destination register (Rd).

Rd → Register destination

Rn → 1st Register

Rm → 2nd Register

- ❖ (6-7 bits) Opcode: 00
- ❖ (4-5 bits) Rd
- ❖ (2-3 bits) Rn
- ❖ (0-1 bits) Rm

Example: ADD X0, X1, X2 → 00000110

SUB Rd, Rn, Rm

Subtracts 2 registers (Rm from Rn) and the difference gets stored in the destination register (Rd).

Rd → Register destination

Rn → 1st Register

Rm → 2nd Register

- ❖ (6-7 bits) Opcode: 01
- ❖ (4-5 bits) Rd
- ❖ (2-3 bits) Rn
- ❖ (0-1 bits) Rm

Example: SUB X0, X1, X2 → 01000110

LDR Rt, Rn, Rm

Loads the data at the address stored in Rn offsetted by Rm to the target register Rt

Rt → Target Register

Rn → 1st Register

Rm → 2nd Register (Offset Register)

- ❖ (6-7 bits) Opcode: 10
- ❖ (4-5 bits) Rt
- ❖ (2-3 bits) Rn
- ❖ (0-1 bits) Offset Register (Rm)

Example: LDR X3, X1, X0 → 10110100

STR Rt, Rn, Rm

Stores the data in Rt at the address stored in Rn offsetted by Rm.

Rt → Target Register

Rn → 1st Register

Rm → 2nd Register (Offset Register)

- ❖ (6-7 bits) Opcode: 11
- ❖ (4-5 bits) Rt
- ❖ (2-3 bits) Rn
- ❖ (0-1 bits) Offset Register (Rm)

Example: STR X0, X2, X1 → 11001001

- Opcode: we only have 2 bits for opcode because we only have 4 instructions
- We used 2 bits per register because we only have 4 general registers. Then we used up to 3 registers per instruction so overall 8 bits per instruction.